

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ДЕРЖАВНИЙ ВИЩИЙ
НАВЧАЛЬНИЙ ЗАКЛАД “УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ” ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ КАФЕДРА
КОМП’ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ

до лабораторної роботи №6

з дисципліни “Програмування мікроконтролерів”

Студента 2-го курсу

Спеціальності 123- “Комп’ютерна інженерія”

Скоробогатого Максима Максимовича

м. Ужгород – 2026 р.

ЛАБОРАТОРНА РОБОТА №6

Мета: Розробка власного комплексного проєкту на основі інтеграції периферійних модулів (матрична клавіатура, 7-сегментний індикатор, RGB-світлодіод, таймери) з використанням архітектури скінченного автомата.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

1. Копіюємо проєкт Лабораторної роботи №5.

До нашої схеми додамо три піни **Digital Output**, назовемо їх *LED_R*, *LED_G*, *LED_B* (див. рис. 1).

Після цього виконаємо налаштування пінів:

- Name: *LED_R* / *LED_G* / *LED_B*
- Type: *Digital Output*
- Drive mode: *Strong drive*
- Initial drive state: *High (1)*

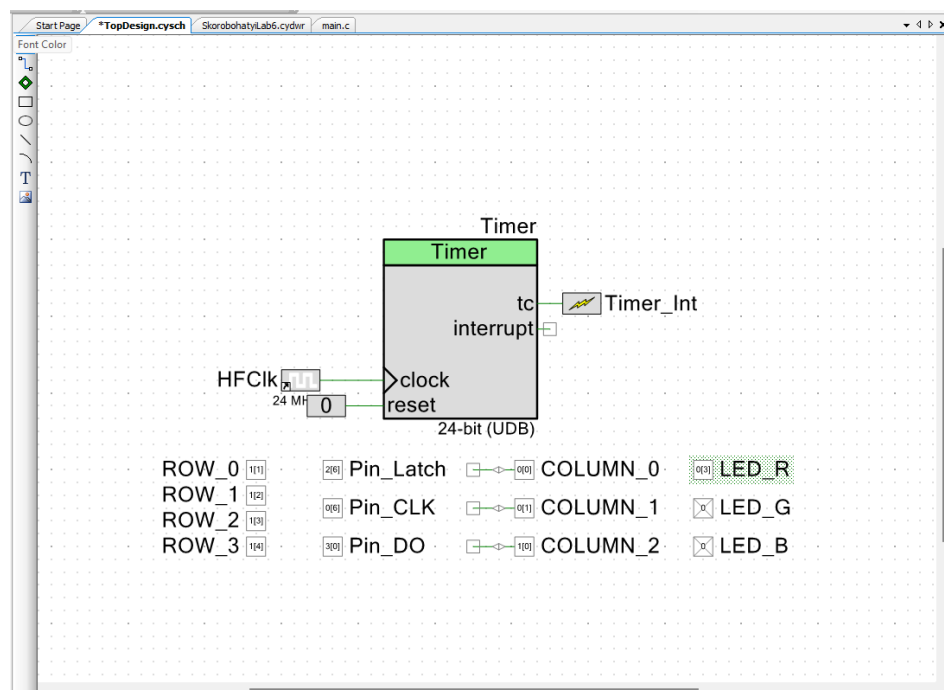


Рисунок 1 – Модифікація проєкту Лабораторної роботи №5

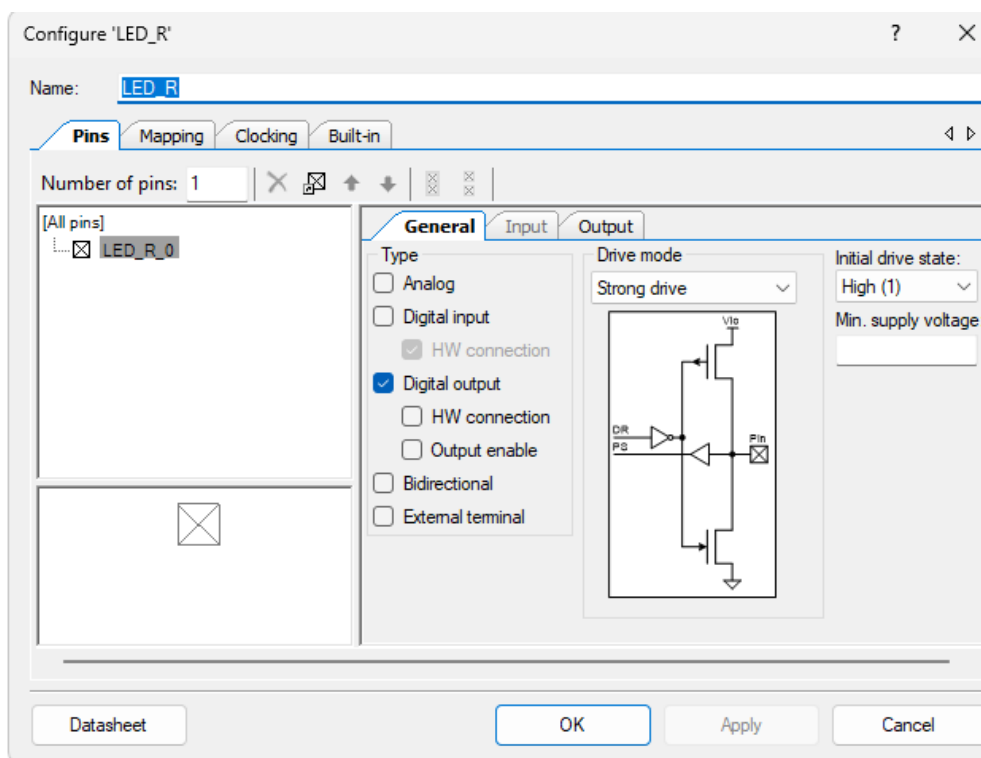


Рисунок 2 – Конфігурація пінів

2. Підключимо наші піни (рис. 3) в відповідності до схеми на платі

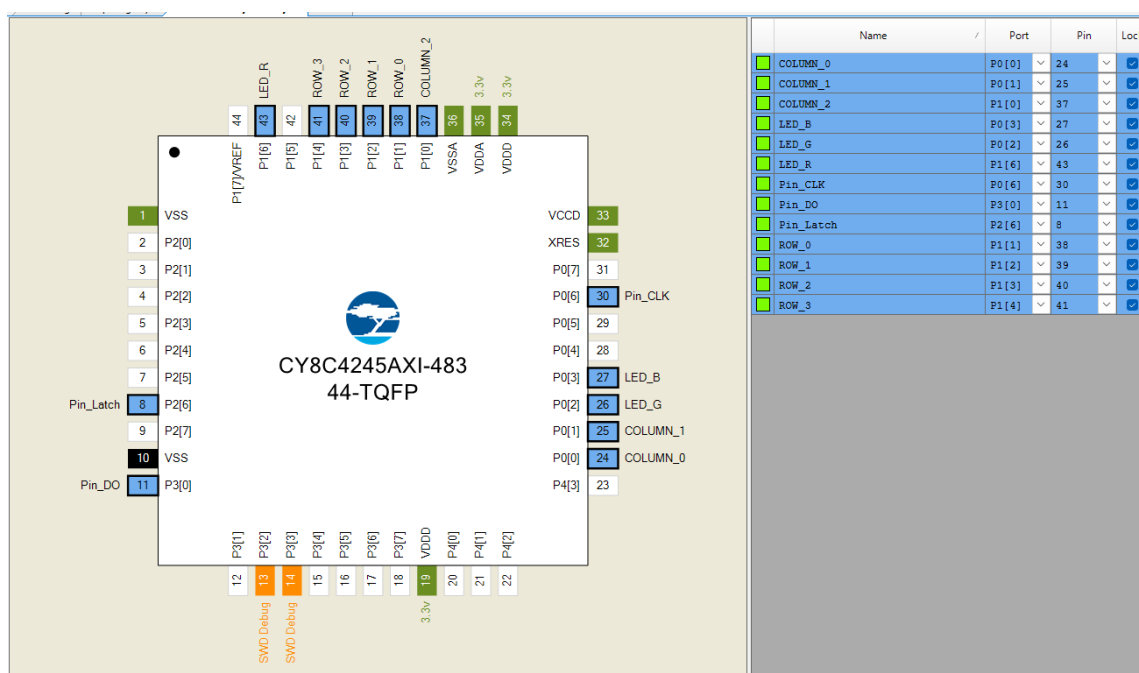


Рисунок 3 – Підключення пінів

3. Опис алгоритму роботи та програмної реалізації

У даній лабораторній роботі було створено комплексний пристрій **«Розумний сейф»**, у якому всі 5 заданих режимів (кодовий замок, таймер, дзеркальний текст, секундомір та рухомий рядок) об'єднані в єдиний логічний сценарій.

3.1. Користувацький сценарій роботи пристрою

Логіка взаємодії користувача з мікроконтролером побудована у вигляді послідовних кроків:

1. **Блокування та очікування пароля:** При старті пристрій заблоковано, RGB-світлодіод світиться червоним кольором. На 7-сегментному індикаторі відображаються прочерки. Користувач має ввести 4-значний PIN-код за допомогою матричної клавіатури.
2. **Штрафний таймер:** Якщо введено неправильний пароль, система блокує подальше введення та запускає зворотний відлік на 15 секунд. На дисплеї з'являється індикатор «Р» (Penalty) та залишок часу. Після закінчення таймера система знову дозволяє ввести пароль.
3. **Анімація відкриття:** Якщо введено правильний пароль («2026»), світлодіод змінює колір на зелений. На екрані відтворюється анімація: слово «OPEN» дзеркально розсувається від центру дисплея до його країв, імітуючи відкриття замка.
4. **Секундомір відкритого стану:** Одразу після завершення анімації автоматично запускається секундомір. Він фіксує точний час (у хвилинах, секундах та десятих частках), протягом якого сейф залишається відкритим.
5. **Захищена нотатка (Рухомий рядок):** Знаходячись у розблокованому стані, користувач може натиснути клавішу «9», щоб перейти у режим нотаток (світлодіод стає жовтим). Після введення до 4 цифр та

натискання клавіші «#», цей текст починає циклічно прокручуватися по екрану (ефект рухомого рядка).

6. **Екстрене блокування:** На будь-якому етапі роботи натискання клавіші «*» миттєво очищує всі дані та повністю блокує пристрій, повертаючи його до першого кроку.

3.2. Програмна реалізація та архітектура

Апаратні переривання (Таймер):

Щоб дисплей, клавіатура та внутрішній годинник працювали паралельно і без затримок, було налаштовано переривання `Timer_Int_Handler`. Воно спрацьовує з високою частотою і виконує два критичних завдання: по-перше, забезпечує динамічну індикацію 7-сегментного дисплея (почергове увімкнення розрядів); по-друге, інкрементує глобальну змінну `system_tick_ms`, яка слугує загальним системним годинником.

Керування станами (FSM):

Головний цикл програми (`main`) не використовує блокуючих затримок на зразок `CyDelay()` для відліку часу. Натомість використовується перемикач `switch(current_state)`, який містить 5 станів: `STATE_LOCKED`, `STATE_PENALTY`, `STATE_OPEN_ANIM`, `STATE_STOPWATCH` та `STATE_MEMO`.

Логіка переходів:

- У стані `STATE_LOCKED` програма зчитує масив натискань і порівнює його з еталоном за допомогою функції `strcmp`. Залежно від результату, автомат перемикає стан на `STATE_OPEN_ANIM` або `STATE_PENALTY`.
- Будь-які часові події (штрафний таймер, швидкість зсуву рухомого рядка, секундомір) реалізовані через порівняння поточного часу `system_tick_ms` із

часом останньої дії `last_action_ms`. Це дозволяє програмі безперервно опитувати клавіатуру, навіть коли на екрані відтворюється анімація.

- Ефект дзеркального тексту реалізовано шляхом симетричного запису символів у ліву (`display_data[i]`) та праву (`display_data[7 - i]`) половини візуального буфера.

Лістинг коду

```
#include "project.h"
#include <stdbool.h>
#include <stdint.h>
#include <string.h>

#define ROWS 4
#define COLS 3
#define DISPLAY_SIZE 8
#define BLANK 0xFF

const uint8_t ascii_font[128] = {
    ['0']=0xC0, ['1']=0xF9, ['2']=0xA4, ['3']=0xB0,
    ['4']=0x99, ['5']=0x92, ['6']=0x82, ['7']=0xF8,
    ['8']=0x80, ['9']=0x90, ['P']=0x8C, ['O']=0xC0,
    ['E']=0x86, ['N']=0xAB, ['-']=0xBF, [' ']=0xFF, ['_']=0xF7
};

typedef enum {
    STATE_LOCKED,
    STATE_PENALTY,
    STATE_OPEN_ANIM,
    STATE_STOPWATCH,
    STATE_MEMO
} safe_state_t;

volatile uint8_t display_data[DISPLAY_SIZE] = {BLANK, BLANK, BLANK, BLANK,
BLANK, BLANK, BLANK, BLANK};
volatile uint32_t system_tick_ms = 0;

static const char key_map[ROWS][COLS] = {
    {'1','2','3'}, {'4','5','6'}, {'7','8','9'}, {'*','0','#'}
};

static uint8_t last_keys[ROWS][COLS];

void HC595_sendByte(uint8_t data) {
    for(uint8_t i = 0; i < 8; i++) {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

CY_ISR(Timer_Int_Handler) {
    static uint8_t digit_idx = 0;
    HC595_sendByte((uint8_t)~(1u << digit_idx));
    HC595_sendByte(display_data[digit_idx]);
```

```

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);

    digit_idx++;
    if(digit_idx >= DISPLAY_SIZE) digit_idx = 0;
    system_tick_ms++;
}

void rgb_set(uint8_t r, uint8_t g, uint8_t b) {
    LED_R_Write(r); LED_G_Write(g); LED_B_Write(b);
}

void clear_display(void) {
    for(uint8_t i = 0; i < DISPLAY_SIZE; i++) display_data[i] = BLANK;
}

void matrix_init(void) {
    COLUMN_0_SetDriveMode(COLUMN_0_DM_DIG_HIZ);
    COLUMN_1_SetDriveMode(COLUMN_1_DM_DIG_HIZ);
    COLUMN_2_SetDriveMode(COLUMN_2_DM_DIG_HIZ);
    for(uint8_t r = 0; r < ROWS; r++) {
        for(uint8_t c = 0; c < COLS; c++) last_keys[r][c] = 1;
    }
}

char get_key(void) {
    char pressed = 0;
    for(uint8_t c = 0; c < COLS; c++) {
        if(c==0) { COLUMN_0_SetDriveMode(COLUMN_0_DM_STRONG);
COLUMN_0_Write(0); }
        else if(c==1) { COLUMN_1_SetDriveMode(COLUMN_1_DM_STRONG);
COLUMN_1_Write(0); }
        else if(c==2) { COLUMN_2_SetDriveMode(COLUMN_2_DM_STRONG);
COLUMN_2_Write(0); }

        CyDelayUs(20);
        uint8_t rows[ROWS] = {ROW_0_Read(), ROW_1_Read(), ROW_2_Read(),
ROW_3_Read()};

        for(uint8_t r = 0; r < ROWS; r++) {
            if((rows[r] == 0) && (last_keys[r][c] == 1)) pressed =
key_map[r][c];
            last_keys[r][c] = rows[r];
        }

        if(c==0) { COLUMN_0_Write(1);
COLUMN_0_SetDriveMode(COLUMN_0_DM_DIG_HIZ); }
        else if(c==1) { COLUMN_1_Write(1);
COLUMN_1_SetDriveMode(COLUMN_1_DM_DIG_HIZ); }
        else if(c==2) { COLUMN_2_Write(1);
COLUMN_2_SetDriveMode(COLUMN_2_DM_DIG_HIZ); }
    }
    return pressed;
}

int main(void) {
    CyGlobalIntEnable;
    Timer_Start();
    Timer_Int_StartEx(Timer_Int_Handler);
    matrix_init();

    safe_state_t current_state = STATE_LOCKED;

    uint32_t last_action_ms = 0;

```

```

uint32_t action_timer_ms = 0;

char pin_buf[5] = {0};
uint8_t input_idx = 0;
uint8_t scroll_step = 0;
bool is_scrolling = false;

rgb_set(1, 1, 1);

for(;;) {
    char key = get_key();

    if(key == '*') {
        current_state = STATE_LOCKED;
        input_idx = 0;
        memset(pin_buf, 0, sizeof(pin_buf));
        clear_display();
        rgb_set(0, 1, 1);
    }

    switch(current_state) {

        // ПАОЛЪ
        case STATE_LOCKED:
            rgb_set(0, 1, 1);
            clear_display();

            for(uint8_t i = 0; i < 4; i++) {
                if(i < input_idx) display_data[i] =
ascii_font[(uint8_t)pin_buf[i]];
                else display_data[i] = 0xBF;
            }

            if((key >= '0') && (key <= '9') && (input_idx < 4)) {
                pin_buf[input_idx++] = key;
            }

            if((key == '#') && (input_idx == 4)) {
                if(strncmp(pin_buf, "2026", 4) == 0) {
                    current_state = STATE_OPEN_ANIM;
                    scroll_step = 0;
                    last_action_ms = system_tick_ms;
                } else {
                    current_state = STATE_PENALTY;
                    action_timer_ms = 15000;
                    last_action_ms = system_tick_ms;
                }
                input_idx = 0;
            }
            break;

        // ТАЙМЕР
        case STATE_PENALTY:
            clear_display();
            rgb_set(0, 1, 1);

            if((system_tick_ms - last_action_ms) >= 1000) {
                if(action_timer_ms >= 1000) action_timer_ms -= 1000;
                else current_state = STATE_LOCKED;
                last_action_ms = system_tick_ms;
            }

            uint8_t sec = action_timer_ms / 1000;
            display_data[0] = ascii_font['P'];

```



```

        display_data[2] = ascii_font[sec / 10 + '0'];
        display_data[3] = ascii_font[sec % 10 + '0'];
        break;

// ДЗЕРКАЛЬНИЙ ТЕКСТ
case STATE_OPEN_ANIM:
    clear_display();
    rgb_set(1, 0, 1);

    const char* txt = "OPE      ";
    if((system_tick_ms - last_action_ms) >= 300) {
        scroll_step++;
        if(scroll_step >= 6) {
            current_state = STATE_STOPWATCH;
            action_timer_ms = 0;
            last_action_ms = system_tick_ms;
        }
        last_action_ms = system_tick_ms;
    }

    for(uint8_t i = 0; i < 4; i++) {
        uint8_t symbol = ascii_font[(uint8_t)txt[(scroll_step + i)
% 8]];

        display_data[i] = symbol;
        display_data[7 - i] = symbol;
    }
    break;

// СЕКУНДОМЕР
case STATE_STOPWATCH:
    clear_display();
    rgb_set(1, 0, 1);

    if(key == '9') {
        current_state = STATE_MEMO;
        input_idx = 0;
        is_scrolling = false;
        memset(pin_buf, 0, sizeof(pin_buf));
    }

    if((system_tick_ms - last_action_ms) >= 10) {
        action_timer_ms += 10;
        last_action_ms = system_tick_ms;
    }

    uint32_t t_sec = action_timer_ms / 1000;
    display_data[4] = ascii_font[(t_sec / 60) % 10 + '0'] & 0x7F;
    display_data[5] = ascii_font[(t_sec % 60) / 10 + '0'];
    display_data[6] = ascii_font[(t_sec % 60) % 10 + '0'] & 0x7F;
    display_data[7] = ascii_font[((action_timer_ms % 1000) / 100)
+ '0'];

    break;

// РУХОМИЙ РЯДОК
case STATE_MEMO:
    clear_display();
    rgb_set(1, 1, 0);

    if((key >= '0') && (key <= '9') && (input_idx < 4)) {
        pin_buf[input_idx++] = key;
    }
    if(key == '#') {
        is_scrolling = !is_scrolling;
        last_action_ms = system_tick_ms;
    }

```

```

    }

    if(!is_scrolling) {
        for(uint8_t i = 0; i < 4; i++) {
            if(i < input_idx) display_data[i] =
ascii_font[(uint8_t)pin_buf[i]];
        }
    } else {
        if((system_tick_ms - last_action_ms) >= 350) {
            scroll_step++;
            if(scroll_step >= 8) scroll_step = 0;
            last_action_ms = system_tick_ms;
        }
        for(uint8_t i = 0; i < 8; i++) {
            display_data[i] =
ascii_font[(uint8_t)pin_buf[(scroll_step + i) % 4]];
        }
        break;
    }
    CyDelay(10);
}
}

```

Висновки

Під час виконання лабораторної роботи було успішно розроблено та протестовано комплексний проєкт «Розумний сейф» на базі мікроконтролера Cypress PSoC 4. Головним здобутком стало застосування архітектури скінченного автомата, що дозволило логічно об'єднати п'ять різних режимів (кодовий замок, штрафний таймер, анімацію, секундомір та рухомий рядок) у єдиний безперервний сценарій. На практиці було закріплено навички конфігурації та синхронізації периферійних модулів: матричної клавіатури, RGB-світлодіода та зсувного регістра для 7-сегментного дисплея. Завдяки використанню апаратних переривань таймера вдалося реалізувати плавну динамічну індикацію та точний відлік часу без застосування блокуючих затримок у коді. В результаті створено стабільну вбудовану систему, яка миттєво реагує на дії користувача та надійно обробляє алгоритми доступу.